

Specification for: Gateway-Enforced Symmetric Intent Authentication (GESIA)

Version: 2.0

Design Philosophy: mRNP-Inspired, Payload-Blind, Symmetric Security, Builder-Enforced, Client-Enforced Identity

1. Core Architecture

GESIA v2 follows a Three-Tier Security Model based on a client-derived symmetric key (Ks) established via a secure handshake:

- **Secure Tunnel (Transport):** Standard TLS provides foundational encryption.
- **Interceptor (Client-Side):** Programmable proxy (e.g., Service Worker) that signs request metadata at creation using Ks (derived from Fingerprint).
- **Validator (Server-Side):** High-performance gateway that verifies **signature and TTL** before any business logic or routing.

Design emphasis:

Client-enforced identity — the client generates ephemeral gatepasses; the server validates **via signature and TTL**, implicitly enforcing Fingerprint binding without separately verifying it.

2. Technical Implementation Details

A. Handshake (Key Exchange)

- **Generation:** Ks derived independently on client and server using ECDH (P-256) or future curves and the client Fingerprint.
- **Fingerprint Binding:** Derived via PayOffFP, combining static (GPU, CPU, API) and dynamic (Canvas, Audio) anchors.
- **Persistence:** Ks stored in Local Cache (Client) and Global Cache (Server), mapped to Key ID (KID).
- **Reactive Key Refresh:** On 401 Unauthorized or mutation, the Interceptor triggers a new handshake and retries pending requests.
- **Session Scoping:** Keys are short-lived (TTL) with overlapping validity to prevent race conditions during environment changes.

Key Derivation Formula:

```
Ks = HMAC(Secret, Fingerprint || Default_TTL || Session_Nonce)
```

- Secret: One-time handshake secret
- Default_TTL: Hidden constant to reduce timing attack exposure
- Session_Nonce: Optional per-session entropy

Note: Once Ks is derived from the Fingerprint, **signature verification is sufficient to enforce Fingerprint binding**. No separate per-message Fingerprint verification is needed.

B. Header Structure

Each message carries a self-destructing header:

Field	Description
kid	Key ID for Ks
v	Protocol version (v2.0)
ts	Unix timestamp (seconds) at message creation
ttl	Message lifespan (e.g., 500ms)
nonce	Random high-entropy string (≥ 128 -bit)
sig	HMAC-SHA256 over Payload

Example Header (JSON $\rightarrow$ Base64):

```
{
  "kid": "abc123",
  "v": "v2.0",
  "ts": 1738502400,
  "ttl": 500,
  "nonce": "7b9f2c1a8e3f4d6b",
  "sig": "f2a7b1c4d3e8..."
}
```

JSON Schema for Implementation:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "GESIA Header",
  "type": "object",
  "properties": {
    "kid": {"type": "string"},
    "v": {"type": "string", "enum": ["v2.0"]},
    "ts": {"type": "integer", "minimum": 0},
    "ttl": {"type": "integer", "minimum": 1},
    "nonce": {"type": "string"},
    "sig": {"type": "string"}
  },
}
```

```
"required": ["kid", "v", "ts", "ttl", "nonce", "sig"]
}
```

C. Metadata Signature & Ephemeral Gatepass

- **Fingerprint Formula:** `Fingerprint = PayOffFP derived identity (hardware + environment)`
- **Signature Formula:** `Signature = HMAC(Ks, Payload || ts || ttl)`

Field Requirements:

Field	Requirement
Payload	Full request body or metadata-only (streaming-first)
ts	Unix epoch timestamp (seconds)
ttl	Millisecond lifespan
nonce	Unique, high-entropy per request
Ks	Derived per client via Fingerprint
sig	HMAC-SHA256 over Payload

Key Differences vs Previous Versions:

- Metadata-only signing preserves streaming efficiency
- Ks derivation binds to Fingerprint — stolen keys are useless on another device
- TTL ensures natural expiration, limiting replay and delayed delivery risks

Note: HMAC signature alone enforces Fingerprint binding; no explicit Fingerprint check is required at the gateway per message.

D. Payload-Blind Trade-off

- Performance-first: avoids body hashing for high-throughput systems
 - Compatible with streaming environments (e.g., Java 8)
 - Ensures structural & intent integrity even if payload is untrusted
 - Extensible: optional body hash profiles for sensitive routes
-

3. Comprehensive Security & Threat Analysis

3.1 Cryptographic Foundation

- Ks derived via ECDH (or future curves)
- HMAC-SHA256 for authentication
- TLS provides transport confidentiality

Risk Surface: Protocol semantics, Fingerprint integrity, TTL enforcement, operational correctness

3.2 Threat Scenarios & Mitigations

Scenario	Mitigation / Notes
Equal-Length Payload Mutation	Structural integrity enforced; optional body hash for sensitive routes
Replay Flooding	TTL + nonce; bounded caches; rate-limiting; distributed Bloom filters
Clock Desynchronization	Strict NTP; server-time bootstrap
Header Stripping	Hard reject requests missing X-GESIA header
Key Rotation Race	Overlapping TTL; reactive handshake handles expired keys

3.3 Stable & Entropic Fingerprint

- Deterministic, tamper-resistant, multi-instance safe
- Optional logical operation IDs for auditing

3.4 Why Not Signed URLs?

- Paths mutable — fragile signing
- Fingerprint + Ks + metadata ensures deterministic verification

4. Extensions & Self-Healing

- Stateless replay defense via TTL + nonce
- Reactive key refresh on mutation or 401 Unauthorized
- Strict length boundary enforcement

5. Non-Negotiable Invariants

1. Version binding mandatory
2. Hard rejection on missing headers or nonce reuse
3. Timestamp always included in HMAC

- 4. Signature verified before application logic
- 5. Short-lived session keys with overlapping TTL
- 6. Fingerprint binding independent of URL/path (enforced implicitly via Ks)

6. Evaluation & Impact Matrix

Category	Assessment	Notes
Performance	High	Metadata-only signing; HMAC hardware accelerated
Streaming	Excellent	Zero stream exhaustion
Scalability	Moderate	Shared cache required; distributed Bloom filters recommended
Integrity	Structural	Intent & method verified; payload optional
Overall	Advanced	Client + builder enforced; ephemeral; environment-bound

7. Strategic Positioning & Roadmap

- Provides structural intent integrity
- Does not replace TLS
- Does not provide non-repudiation

Future Evolution Table:

Feature	Purpose
X25519 support	Modern curve alignment
Optional Body Hash Profile	High-sensitivity route integrity
Distributed Bloom Filters	Global replay defense

Operational Guidance:

- Strict NTP synchronization
- Bounded nonce caches
- Enforce TTL and overlapping key windows
- Audit logging of rejected requests
- Optional features version-bound

8. Environmental Sovereignty via Client-Derived Keys

8.1 Architectural Philosophy: The Digital Cell

- Server = Nuclear Pore; Client = Nucleus
- Messages carry their own physics (TTL) and DNA (Fingerprint)
- Zero-assumption scaling: Gateway re-calculates truth
- Metabolic decay: rapid expiration of messages and keys

8.2 Component Specifications

A. Fingerprint (Anchor):

- Generated via PayOffFP (static: GPU, CPU, API; dynamic: Canvas, Audio)
- Salt for key derivation; shifts on environment change (“mutation”)

B. Key Derivation (Ks):

- Derived per-client using Fingerprint and session nonce
- Ephemeral, context-bound, never transmitted

C. Self-Destructing Packet (mRNP):

- Header contains ts, ttl, and HMAC(Payload || ts || ttl)
- Messages expire naturally when TTL exceeded

8.3 Lifecycle of a Request

```

Client (Interceptor)
  |
  |-- sign payload with Ks (Fingerprint-bound)
  v
Network (Cytoplasm)
  |-- message TTL ticking
  v
Gateway (Validator)
  |-- enforce TTL
  |-- verify HMAC signature (implicitly verifies Fingerprint)
  |-- proceed to routing/application logic

```

8.4 Mutation Handling

State	Action
Detection	Interceptor sees Fingerprint change
Re-Agreement	Client sends mutation event
Grace Period	Gateway maintains old key for $1 \times \text{TTL}$
Apoptosis	Old Fingerprint/Key purged after TTL

8.5 Security Gains

- Anti-replay via TTL + nonce
 - Anti-theft via environment-bound Ks
 - Stateless scaling; server only needs Secret + Agreement Cache
 - **No per-message Fingerprint verification required**
-

9. Design Sophistication & Purposeful Primitives

- Ks: Derived, ephemeral, environment-bound
- HMAC Anchoring: Metadata + Fingerprint binding
- Timestamp & TTL: Birth and metabolic decay of messages
- Fingerprint Anchoring: Client-enforced identity; mutation handling
- Nonce & Metadata: Entropy, anti-replay, lightweight verification
- Ephemeral Gatepass: Self-contained, verifiable message unit; extensible

Innovation: Biologically inspired lifecycle; client-as-entity paradigm; self-regulating ephemeral authentication

10. Nonce / Replay Cache Policy

- Server maintains bounded cache of recent nonces
 - Duplicate nonce → hard reject
 - Cache duration = TTL + safety margin (recommended: $1 \times \text{TTL}$)
 - Optional distributed Bloom filters for multi-node deployment
-

11. Key Refresh & Mutation Timeline

- Key TTL: default 500ms (tunable)
 - Overlapping TTL windows ensure old key validity for in-flight messages
 - Environment mutation triggers reactive handshake
 - Grace period = $1 \times \text{TTL}$ for delayed packets
-

12. Appendix: Implementation Guidance

- TLS mandatory — GESIA does not replace transport encryption

- Fingerprint verification only required during handshake
 - Reactive key refresh must atomically handle pending requests
 - Optional invariants must be version-bound
 - TTL reduces exposure; handshake frequency must account for operational load
 - Audit logging is critical
-

GESIA v2 Summary

- Fully user-controlled, builder + client-enforced primitive
- Payload-blind, metadata-signed for high performance
- TTL + timestamp enforce ephemeral self-destruct semantics
- Fingerprint-bound Ks ensures environment-specific security
- Self-healing, reactive, operationally resilient
- Extensible, disciplined, auditable, and future-ready